Cross-Device Consistency in Flutter: Strategies and Solutions for Achieving Optimal User Experience Across Diverse Platforms.

Luana Kimley
D00234604@student.dkit.ie

Abstract: This research paper explores the effectiveness of Flutter in ensuring a seamless user experience across diverse devices. Focusing on challenges related to screen sizes, device orientation, and platforms, the study investigates Flutter's capabilities. It reveals key strategies employed by developers to enhance user experiences, encompassing design principles, layout responsiveness, and adaptive widgets. Through a blend of theoretical exploration and practical insights, this study provides actionable guidance for developers aiming to optimize cross-device consistency and elevate overall user satisfaction in Flutter development.

Keywords: Flutter development, responsive design framework, cross-device consistency.

1. Introduction

In our Collaborative Project, I am a member of team Pesado, a team consisting of five people developing an injury prevention and organisation app for GAA teams called "PlayMetrix". Our main users are the manager, player, physio, and coach of a team. Each of the users have different roles in the application, depending on what their role is in the team. Managers are responsible for setting up a team, adding training and matchday schedules along with announcements, editing player availability, and recording player match time. Players would be able to view their injury history, availability, schedules, statistics, and edit their own profiles. Physios are allowed to add injuries to a player, view their injuries, and edit their availability. Coaches would be able to view players' injury history and profile and also post announcements related to schedules.

My roles in this project are as the Frontend Developer and UI/UX Designer. The main role I will investigate here is my role as the Frontend Developer. As a Frontend Developer, my task is to develop the user interface of our application using Flutter, an open source framework by Google for building beautiful, natively compiled, multi-platform applications from a single codebase [1]. While developing the application, I only tested my application through a single simulator through my laptop and did not consider other devices as it did not cross my mind that it would look different on other screens. I only realised this after we did some frontend testing on different devices, the screens look different cross device, considering factors such as the platform, device orientation, and screen sizes.

2. Research Question

The initial research question, "How can Flutter ensure consistent user experience across different devices?" underwent refinement to more precisely investigate the framework's mechanism for achieving cross-device consistency. The refined question, "How does Flutter address challenges in achieving cross-device consistency, and what are the key strategies for optimal user experience across diverse devices?" introduces a focused lens on Flutter's cross-platform design principles.

This modification recognises the importance of responsive design in addressing challenges related to different screen sizes, device orientation, and platforms. The refined question explores specific tools, widgets, and strategies in Flutter that foster a seamless user experience across devices. By explicitly addressing challenges and highlighting key strategies, the refined research question guides the inquiry towards actionable insights for developers aiming to improve cross-device consistency in their Flutter applications.

3. Justification of Importance

To answer the research question, it is essential to understand the challenges faced by Flutter developers in maintaining cross-device consistency. These challenges arise from the platform's flexibility, diverse device specifications, and the delicate balance between customisation and standardised user experience.

Flutter follows an open and layered architecture, which means that developers can make their own widgets at any level of the layered architecture [2]. This flexibility is advantageous for creativity, but it may pose challenges in maintaining a consistent user experience across different devices. Developers might introduce variations that impact cross-device consistency, such as custom layouts for specific screens that may not adapt well to different screen sizes and orientations, causing layout issues and impacting the overall user experience.

The challenge with Flutter lies in its departure from native system components, offering a unique set of widgets for UI rendering within a single code base [3]. This approach enables code reuse between iOS and Android but demands careful attention from developers to ensure consistent app design, branding, and adherence to platform-specific guidelines [4]. The primary hurdles in Flutter development revolve around harmonising in-app navigation patterns, native UI components, and micro-interactions/animations across Android and iOS, posing challenges for developers striving to maintain a cohesive user experience across different platforms [5].

4. Proposed Solutions

In order to address the challenges above, developers working with Flutter can implement a set of strategies and best practices.

To tackle the variability in custom layouts, an emphasis on responsive design principles is crucial. Developers should adopt flexible layouts that dynamically adjust to diverse screen sizes and orientations. Flutter allows developers to do this by using two basic approaches, which are the **LayoutBuilder** class and the **MediaQuery.of()** method, along with other useful widgets and classes, which are **AspectRatio**, **CustomSingleChildLayout**, **CustomMultiChildLayout**, **FittedBox**, **FractionallySizedBox**, **MediaQuery**, **MediaQueryData**, and **OrientationBuilder** [6].

Given Flutter's departure from native system components, leveraging the framework's adherence to Material Design guidelines becomes a key solution. Developers can utilise Flutter's theming capabilities to customise the app's appearance while ensuring consistency with both Android and iOS design principles. Flutter is compatible with Google's design framework, Material Design, which guarantees a uniform and attractive

user interface across devices [7]. This approach facilitates the creation of a unified design language, aligning with Material Design for a cohesive visual experience.

In summary, tackling challenges in Flutter development requires a multifaceted approach, involving responsive layouts, theming, and adherence to design guidelines. By integrating these strategies, developers can navigate complexities and ensure a harmonized user experience across diverse devices and platforms.

5. Research Method

In undertaking this applied research, a structured approach will be followed to ensure a systematic progress and meaningful outcomes. The proposed steps are outlined below:

1. **Research Planning and Design**: Develop a detailed research plan, identifying specific pages within the app based on the tester's feedback sheet. Formulating clear objectives and success criteria for the research is also essential. This will require approximately 2 hours.
2. **Code Modification and Implementations**: Review the proposed solutions and adapt them to address the identified challenges on the selected pages. Implement the modifications systematically, making sure to align with the research objectives. This will require approximately 8 hours.
3. **Comprehensive Testing**: After the initial implementation, comprehensive testing with various designs, screen sizes, and orientations will be done. The step involves rigorous testing to validate the effectiveness of the code modifications and identify any potential issues that may have arisen during the implementation phase. This will require approximately 3 hours.

The collected data will include insights on the effectiveness of the proposed solutions in achieving cross-device consistency through screenshots of the screens of the app within various devices before and after modifying the code to adhere with the proposed solutions.

6. Results

In the initial phase of the research, specific pages within the app were identified based on a thorough review of the tester's feedback sheet. The key components of the identification process included:

- **Pages with Failed Tests**: Only pages that did not pass the functionality test will be selected for evaluation. This indicates that the pages are not compatible cross-devices and needs further evaluation and enhancements.
- **Severity Assessment**: Evaluate the severity of issued reported on each page to prioritise interventions.
- **Common Themes or Patterns**: Identify any recurring themes or patterns across multiple pages to address systemic issues rather than isolated incidents.

The research objectives of this research are:

- Identify and address frontend issues within the app based on the feedback sheet provided by the tester.
- Implement proposed solutions to enhance cross-device consistency.
- Evaluate the effectiveness of the code modifications through comprehensive testing.

The success criteria of the modified pages are:

- Reduction in identified frontend issues by having consistent look and functionality across various devices.

Here are the pages identified for modification, along with the modifications done to fix the issues:

1. **Sign Up – Choose Type Page**

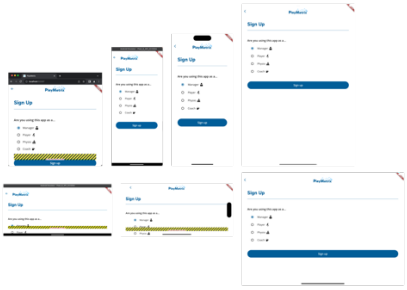

Fig. 1.1. Sign Up – Choose Type page before modifications

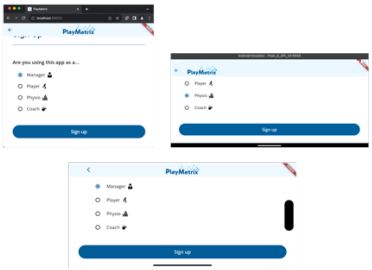

Fig. 1.1. Sign Up – Choose Type page after modifications

2. **Home Page**

Fig. 2.1. Home page before modifications

Fig. 2.2. Home page after modifications

3. **Team Page**

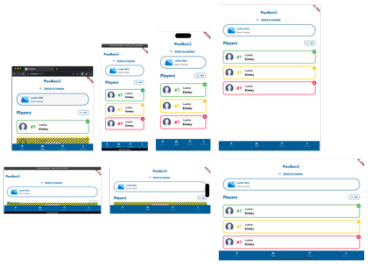

Fig. 3.1. Team page before modifications

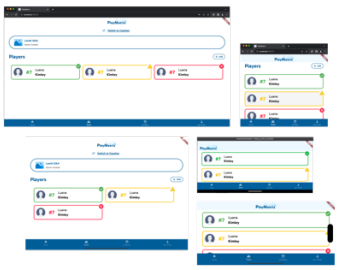

Fig. 3.1. Team page after modifications

The selected pages underwent extensive testing across diverse devices, including a Chrome web browser, Google Pixel 6 emulator, iPhone 15 Pro Max emulator, and iPad 10 emulator, encompassing various screen sizes and orientations (portrait and landscape). Pages displaying a yellow warning indicated overflow, rendering the bottom section inaccessible to users due to the overflow by a specific pixel count.

The pages maintained a consistent appearance and layout across different devices, which is a positive outcome. This uniformity was ensured during the app development phase by incorporating Material Design widgets instead of relying on native components. This strategic choice contributed to a seamless and harmonised user experience across platforms, eliminating any issues in this aspect.

A consistent issue identified across the selected pages was insufficient screen width to accommodate all widgets, particularly noticeable in the Chrome web browser, landscape mode on Google Pixel 6, and landscape mode on iPhone 15 Pro Max. This posed a significant challenge for users, hindering their ability to complete crucial functionalities such as signing up due to the inaccessibility of buttons. Additionally, vital information, such as menus on the home screen and player lists, became inaccessible, making the usability problem worse.

To address the common issue underlying the failing screens, a unified solution was implemented, which is wrapping a **SingleChildScrollView** widget around the existing content. This approach enables users to scroll through content that extends beyond the visible area, resolving the problem consistently across various screens. The code snippet below illustrates the implementation:

```
body: SingleChildScrollView(
    child: Padding(
        padding: const EdgeInsets.only(top: 10, right: 35, left: 35),
        child: Column(children: [
          Row(mainAxisAlignment: MainAxisAlignment.center, children: [
            const Icon(Icons.sync_alt,
                color: AppColours.darkBlue, size: 24), // Icon
            underlineButtonTransparent("Switch to Coaches", () {}),
          ]), // Row
          const SizedBox(height: 10),
```

Fig. 4. Code snippet of the implementation of the **SingleChildScrollView** class

Additionally, the introduction of the **MediaQuery.of()** method addressed concerns raised during user testing on the Teams page. To optimise screen space on wider displays (beyond 1000 pixels), the player profile widgets were designed to dynamically wrap around, accommodating other buttons more efficiently. The code snippet for this can be seen below.

```
if (MediaQuery.of(context).size.longestSide ≥ 1000)
  Wrap(
      direction: Axis.horizontal,
      spacing: 20.0,
      runSpacing: 20.0,
      children: [
        _playerProfilePill(
```

Fig. 5. Code snippet of the implementation of the implementation of the **MediaQuery.of()** method

7. Conclusion

In conclusion, this research systematically addressed cross-device consistency in the PlayMetrix app through meticulous planning, code modifications, and comprehensive testing. Identified pages underwent evaluation based on failed tests, severity assessments, and common themes, with the objective of reducing frontend issues and ensuring consistent functionality across devices. The introduction of a **SingleChildScrollView** widget and **MediaQuery.of()** method proved effective in resolving the recurring issue of screens not accommodating all widgets. This research contributes practical insights into achieving cross-device consistency in Flutter app development, emphasising responsive design principles and strategic code modifications for an optimal user experience.

References

[1] Flutter. (2019). *Flutter – Build apps for any screen* [online]. Available from: https://flutter.dev/ [accessed 22 November 2023].

[2] Dagne, L. (2019). *Flutter for cross-platform App and SDK development* [online]. Bachelor's thesis, Metropolia University of Applied Sciences. Available from: https://www.theseus.fi/bitstream/handle/10024/172866/Lukas%20Dagne%20Thesis.pdf [accessed 25 November 2023].

[3] Szuka, B., Janiec, J. and Bartosińska, I. (2023). *Flutter vs React Native – Which is Better for Your Project?* [online]. Available from: https://www.thedroidsonroids.com/blog/flutter-vs-react-native-comparison [accessed 25 November 2023].

[4] Cheon, Y. and Chavez, C. Converting Android Native Apps to Flutter Cross-Platform Apps. (2021). In: *2021 International Conference on Computational Science and Computational Intelligence (CSCI) 15-17 December, Las Vegas, NV, USA*, pp.1898-1904. Available from: doi: 10.1109/CSCI54926.2021.00355 [accessed 25 November 2023].

[5] Palumbo, D. (2021). *The Flutter Framework: Analysis in a Mobile Enterprise Environment* [online]. Master's thesis, Politecnico di Torino. Available from: https://webthesis.biblio.polito.it/secure/19111/1/tesi.pdf [accessed 25 November 2023].

[6] Flutter. (2023). *Creating responsive and adaptive apps* [online]. Available from: https://docs.flutter.dev/ui/layout/responsive/adaptive-responsive [accessed 26 November 2023].

[7] Sattar, A. M., Soni, P., Ranjan, M. K., Kumar, A., Sahu, C., Saxena, S. and Chaudhari, P. Open Source Developments Accelerating Cross-platform Development with Flutter Framework. (2023). *Journal of Open Source Developments*. 10(2), 10.37591/JoOSD. Available from: https://www.researchgate.net/publication/374628506_Open_Source_Developments_Accelerating_Cross-platform_Development_with_Flutter_Framework [accessed 26 November 2023].